CSI500 Stock Selection via GBDT Ensembles: A Disciplined Path from Naive Complexity to Performance

Jinghan Wu
NYU 2026 Spring, Machine Learning

May 11, 2026

Abstract

We build a long-only stock-selection model on the CSI500 universe with the objective of outperforming the index over a 5 trading-day horizon. The final system is an ensemble of XGBoost and LightGBM regressors trained on approximately 60 engineered cross-sectional factors with a 3-day forward-return target. On a 30-window non-overlapping rolling backtest spanning roughly 2025-09 to 2026-04, the model achieves a cumulative return of **+53.4%** compared to the supplied baseline at **+20.0%** and the CSI500 benchmark at **+23.8%**, a cumulative excess of **+29.6 percentage points** over the index and **+33.4 percentage points** over the baseline. The supplied baseline finishes about 4 percentage points behind CSI500 on this sample, underscoring the challenge of capturing cross-sectional dispersion during the volatile late-2025 to early-2026 regime. Beyond reporting these numbers, this report documents the iterative path that produced the result, including several earlier configurations that, despite being more complex than the baseline, were measurably worse. The lessons drawn from those failures, and the ablation studies that justify the final configuration, are a central contribution of this work.

1 Introduction

The competition task is straightforward to state. At each submission deadline we output a long-only portfolio of CSI500 constituents subject to three mechanical constraints: a minimum of 30 names with non-zero weight, a per-name cap of 10%, and non-negative weights summing to 1. The portfolio is held over a fixed forward window and scored against the CSI500 index by realised excess return.

The challenge is not the constraints themselves. The real difficulty lies in the signal-to-noise ratio of cross-sectional Chinese A-share returns at a weekly horizon. Daily cross-sectional rank Information Coefficients (IC) of stable models in this market are typically in the range of 0.02 to 0.05. Walking from such an IC to a portfolio that reliably outperforms the benchmark requires every ingredient to be calibrated against out-of-sample evidence: features, model, validation methodology, and portfolio construction.

Our final pipeline (Sections 3 and 4) is an ensemble of 15 boosted-tree models trained on roughly 60 technical factors plus 22 cross-sectional rank factors. The factor library draws on common technical-analysis primitives such as K-bar shapes, multi-horizon momentum, volatility, RSI, MACD, and Bollinger position, augmented with extreme-value position factors (IMAX/IMIN) and an Amihud

illiquidity proxy. Predictions are translated into weights through a pure rank weighting scheme, followed by iterative cap redistribution to enforce the 10% per-name limit.

The headline result is that on a 30-window non-overlapping rolling backtest with 5-day holding periods covering 2025-09 to 2026-04, the model delivers a cumulative excess of +29.6 percentage points over CSI500 and +33.4 percentage points over the supplied baseline. We provide two complementary backtests of length 15 and 60 windows in Section 5 as a robustness check.

We also dedicate a substantial portion of the report (Section 6) to configurations we tried and rejected. The most cautionary of these was an early system using a 5-day target which underperformed the final 3-day-target configuration by tens of cumulative percentage points on the same backtest. The path from that earlier failure to the current result is, in our view, the most generalisable contribution of this work.

2 Self-Test Methodology

This section addresses the self-test grading criterion (a) directly. We describe how the data is partitioned into training, validation, and test sets, why the partition is leak-free, and how the test performance is computed. The numerical test result is reported in Section 5, addressing criterion (b).

2.1 Data

We use forward-adjusted daily bars (open, high, low, close, volume, amount, turnover) on the 499 CSI500 constituents for which complete history was available, from 2020-01-02 through 2026-04-30. The CSI500 index series `000905` is used as the benchmark. No alternative data sources are integrated in this version.

2.2 Time-Based Split with Embargo

Random k-fold cross-validation is invalid for this problem. The training target is the 3-day forward return, so any sample at date t uses information from dates $t+1$, $t+2$, $t+3$ in its label. If a validation row shares its target window with a training row's target window, the same future bar enters the model twice: once as a training label and once as a validation label. This leaks the validation distribution into training and inflates IC estimates substantially.

To prevent leakage we use a strict time-based partition with a discarded embargo region between the training and validation periods. For each prediction date t, denoted `as_of`, the partition is defined as follows:

- **Training set.** All dates up to $t - V - E - 1$ trading days, where $V = 10$ is the validation window length and $E = 5$ is the embargo length. The maximum date in the training pool is also capped at $t - H$ where $H = 3$ is the forward horizon, so that no training label can use the price at `as_of`.
- **Embargo region.** The 5 trading days immediately preceding the validation start. These are discarded entirely. They prevent any training label, which uses prices up to 3 days into the future, from reaching into the validation feature window.
- **Validation set.** The 10 trading days immediately preceding the training cutoff. Used for early stopping and for monitoring the validation rank IC.

- **Test set.** The 5 trading days from $t+1$ to $t+5$. The portfolio is built on `as_of` and held through this window. The realised excess return on this window is the test metric.

The choice $E = 5$ is set by `max(5, FORWARD_HORIZON + 2) = max(5, 5) = 5` in the code. Together with the cap on training labels at $t - H$, the construction guarantees that no row in the training pool shares a price bar with the validation period or with the test window.

2.3 Why the Split is Leak-Free

The argument has three pieces. First, the embargo of 5 trading days between training end and validation start exceeds the 3-day forward horizon, so a training label cannot extend into a validation feature day. Second, the training pool is capped at $t - H$, so a training label cannot extend up to or past `as_of` itself. Third, the test window is strictly after `as_of`, and prices in this window are accessed only by the scoring function, never by feature construction or model fitting.

We do not use the validation rank IC for hyperparameter tuning. The validation set serves only for early stopping and as a sanity check during training. All hyperparameters are fixed before the rolling backtest is run, which removes the most common source of subtle leakage between validation and test.

2.4 Rolling Test as Held-Out Evaluation

Because Chinese A-share returns exhibit substantial regime variation across years, a single fixed test window would not be informative about generalisation. We instead run a non-overlapping rolling backtest. Starting from the most recent 5-day window in the data, we step backwards in 5-day increments and re-run the entire pipeline at each `as_of` date. The pipeline includes feature construction, train/validation split, ensemble training, prediction, portfolio construction, and realised-return scoring.

The series of disjoint test-window excess returns produced by this procedure is the basis of every performance number reported in Section 5. Crucially, these test windows are non-overlapping, non-stationary, and never seen by the model during training, so the aggregate excess return is a clean estimate of out-of-sample performance.

Algorithm 1 Rolling 5-day backtest, function `run_self_test`.

Require: panel P, prices, index, top_k, num_windows
$D \leftarrow$ sorted unique trading dates in P
start_idx $\leftarrow \max(\text{warmup}, |D| - 5 \cdot \text{num\_windows} - 5)$
for idx = start_idx **to** $|D| - 5$ **step** 5 **do**
    as_of $\leftarrow D[\text{idx}]$
    (train, val) $\leftarrow$ split with embargo at `as_of`
    models $\leftarrow$ `train_ensemble`(train, val)
    scores $\leftarrow$ `ensemble_predict`(models, $P_{\text{as\_of}}$)
    $w \leftarrow$ `build_portfolio`(scores, top_k)
    $r \leftarrow$ `score_window`(w, prices, $[D[\text{idx}+1], D[\text{idx}+5]]$)
    record $r_{\text{portfolio}}$, $r_{\text{benchmark}}$, r_{excess}
end for
return DataFrame of per-window realised returns

2.5 Validation Metric vs. Test Metric

The validation metric is the per-day cross-sectional Spearman rank correlation between predicted scores and realised 3-day forward returns, averaged across the 10 validation days (validation rank IC). It is informative for early stopping but not directly comparable to the competition's evaluation metric.

The test metric is the realised 5-day excess return of the portfolio relative to CSI500 on a held-out test window, exactly matching the competition's live-evaluation rule in Section 5 of the rules. Reporting the test metric, not the validation IC, is what answers the self-test grading question of whether the model exceeds the baseline.

3 Factors: What Signals We Used and Why

3.1 Design Philosophy

Our factor library is moderately expanded. There are roughly 60 raw factors and 22 cross-sectional rank factors, for approximately 82 inputs to the model. This is substantially richer than the supplied baseline at 14 features, but substantially leaner than full Alpha158-style libraries that contain 150 or more factors. The number is not arbitrary. An earlier version of this work used a less curated 50-factor library together with a 5-day target and underperformed the baseline by approximately 14 cumulative percentage points on a comparable backtest. That experiment is discussed in Section 6.1.

The library is organised around four intuitions, in rough order of incremental signal contribution:

1. **Price-shape primitives (K-bar).** Rather than relying purely on close-to-close returns, the relative positions of open, high, low, and close within a day capture intraday tone. Empirically these factors have low correlation with momentum and add real explanatory power.

2. **Multi-horizon momentum and volatility.** Returns and standard deviations at 1, 2, 3, 5, 10, 20, 30, and 60-day windows. The model learns interactions between short-term reversal and longer-term trend that cannot be captured by any single horizon.

3. **Microstructure proxies.** Volume z-score, 5/20-day volume ratio, turnover moving average, and the Amihud illiquidity proxy. These distinguish names with durable institutional interest from those temporarily inflated by transient retail flows.

4. **Cross-sectional ranks.** For the most informative base factors we additionally compute daily percentile ranks. This neutralises individual-stock distributional differences (some names are structurally more volatile than others) and gives the model a regime-invariant view of each input.

3.2 Factor Library

The complete factor inventory is summarised in Table 1.

Table 1: Factor library: 40 raw factors plus 22 cross-sectional rank factors.

Category	Count	Examples
K-bar shape	9	KMID, KLEN, KMID2, KUP, KLOW, KSFT, ...
Momentum	7	ret_1d, ret_2d, ret_3d, ret_5d, ret_10d, ret_20d, ret_60d
Volatility	5	vol_5d, vol_10d, vol_20d, vol_30d, vol_60d
Volume / Turnover	3	volume_z_20d, vol_ratio_5_20, turnover_ma_20d
MA deviation	2	close_over_ma20, close_over_ma60
RSI	2	rsi_14, rsi_5
Price-Volume corr.	3	pv_corr_5d, pv_corr_20d, pv_corr_60d
Drawdown / Run-up	2	drawdown_20d, runup_20d
MACD / Bollinger	2	macd_hist, bb_pos
Extreme position	3	IMAX20, IMIN20, IMXD20
Tail risk / Liquidity	2	max_drop_10d, amihud_20d
Raw factors total	**40**	
Cross-sectional ranks	**22**	e.g. ret_5d_rank, KMID_rank, IMAX20_rank

3.3 Cross-Sectional Processing

Per-stock factors are first computed as standard time-series rolling quantities. The full panel is then processed daily across all stocks. Two operations are critical.

The first is per-feature winsorisation at the 1st and 99th percentiles, applied independently each day. This reduces the influence of extreme outliers in any single stock-day on tree-split decisions, which is important for GBDT models since extreme points can dominate the gain calculation at split candidates.

The second is target z-scoring with bounded extremes. The 3-day forward return is winsorised then z-scored cross-sectionally per day. This is equivalent to training the model to predict day-relative ranks rather than absolute returns, which is the correct objective when the downstream operation is a top-K selection. The bounded extremes prevent a few outlier days, such as crash sessions or limit-up rallies, from dominating the model's loss surface.

```
def _winsorize(x: pd.Series) -> pd.Series:
    if len(x) < 30 or x.isna().all():
        return x
    lo, hi = x.quantile(0.01), x.quantile(0.99)
    return x.clip(lower=lo, upper=hi)

# Per-feature daily winsorise
grouped = panel.groupby("date")
for f in feat_cols:
    panel[f] = grouped[f].transform(_winsorize)

# Per-feature daily rank (selected base factors only)
for base in RANK_BASE:
    panel[f"{base}_rank"] = grouped[base].rank(method="average", pct=True)

# Target: cross-sectional z-score with bounded extremes
def _zscore_safe(x: pd.Series) -> pd.Series:
    if len(x) < 30 or x.isna().all():
        return x
    x = x.clip(lower=x.quantile(0.01), upper=x.quantile(0.99))
    return (x - x.mean()) / (x.std() + 1e-8)
```

```
22
23 panel[TARGET_COLUMN] = panel.groupby("date")[TARGET_COLUMN].transform(_zscore_safe
       )
```

Listing 1: Cross-sectional winsorisation, rank generation, and target normalisation. The function `_zscore_safe` winsorises target values before z-scoring so that a few tail observations do not dominate the squared-error loss.

4 Models: Training and Portfolio Construction

4.1 Target Variable

We predict the 3-day forward simple return, $r^{(3)}_{i,t} = c_{i,t+3}/c_{i,t} - 1$, even though the live evaluation window is 5 trading days. This counter-intuitive choice is the result of an ablation study described in Section 6.4. The 3-day target empirically yields substantially higher out-of-sample portfolio returns than a 5-day target on the same backtest. Our interpretation is that the shorter-horizon label has a higher signal-to-noise ratio for training, while the rank correlation between 3-day and 5-day forward returns is high enough that the slight horizon mismatch at evaluation does not overwhelm the gain in label quality.

4.2 Ensemble of Boosted-Tree Learners

The final ensemble consists of 15 models obtained from 5 random seeds applied to 3 distinct base configurations: one XGBoost configuration and two LightGBM configurations. Using two GBDT implementations is intentional. XGBoost and LightGBM differ in tree-growth policy (level-wise versus leaf-wise) and in feature-bucketing heuristics. These differences produce predictions whose errors are imperfectly correlated, so the average across the ensemble has lower variance than any single learner.

The full hyperparameters of the three configurations are listed in Table 2. All three use shallow trees (depth 3 to 4) with strong L_1 and L_2 regularisation. This configuration was deliberately conservative. The training set after winsorisation and z-scoring is relatively well-behaved, and shallow regularised trees were less prone to fitting noise during the early-2025 portion of the backtest.

Table 2: Final ensemble configurations. Each configuration is fit 5 times with different random seeds drawn from the set $\{42, 7, 99, 2024, 1234\}$, yielding 15 models in total. Time-decay sample weights with a 120-day half-life are applied to all configurations.

Model	n_{est}	**depth**	**lr**	**subsample**	**colsample**	L_1/L_2
XGB-A	400	3	0.03	0.7	0.7	1.0/2.0
LGB-A	400	3	0.03	0.7	0.7	1.0/2.0
LGB-B	400	4	0.02	0.8	0.6	0.5/3.0

Two technical details made this ensemble effective in practice. The first is time-decay sample weighting. Older training samples receive lower weight via an exponential decay with a half-life of 120 trading days. The market microstructure of A-shares has shifted noticeably across the five-year training window, so very old samples are at risk of containing patterns that no longer hold. The half-life of 120 days was chosen because it produced the most consistent backtest performance among

the values we considered, namely 60, 120, 250, and 500 days, with longer half-lives performing slightly worse.

The second is early stopping. Each model uses the held-out 10-day validation window to stop training at 50 rounds without improvement. This prevents over-fitting on the most recent training period without requiring manual selection of the number of boosting rounds.

```
def make_time_decay_weights(train_df, half_life_days=120):
    if half_life_days <= 0:
        return np.ones(len(train_df), dtype=float)
    dates = pd.to_datetime(train_df["date"])
    age_days = (dates.max() - dates).dt.days.astype(float)
    weights = np.power(0.5, age_days / half_life_days)
    return np.asarray(weights / weights.mean(), dtype=float)
```

Listing 2: Time-decay sample weights with a 120-day half-life. Age is measured in calendar days relative to the most recent training date.

4.3 Portfolio Construction

Given the vector of ensemble-averaged scores at the prediction date, the portfolio is constructed in three steps.

The first step is a liquidity filter. We drop any name with a 20-day mean turnover below 0.05% or with zero current-day volume. This removes suspended names and structurally illiquid issues that the live scoring function would treat poorly.

The second step is top-K selection. The 30 stocks with the highest ensemble z-score are kept and all others are discarded. We use $K = 30$ as the operating point. Section 6.2 discusses the ablation that motivates this choice.

The third step is rank weighting with iterative cap redistribution. Initial weights are assigned in proportion to the rank of each selected stock, so that the highest-ranked name receives a weight proportional to K, the second-ranked receives a weight proportional to $K - 1$, and so on down to weight 1 for the lowest-ranked name in the top K. After normalisation, the highest-ranked stock receives roughly 6.5% of capital and the lowest-ranked roughly 0.2%. Any weight that exceeds the 10% per-name cap is iteratively capped, and the redistributed surplus is reallocated to uncapped stocks proportionally to their current weights. The procedure iterates until all weights are below the cap.

We chose pure rank weighting over a blended scheme on the basis of the ablation reported in Section 6.3. A blended scheme that mixed rank weights with equal weights produced a meaningfully smaller cumulative excess on the same backtest, suggesting that the model's confidence ordering at the top of the cross-section is reliable enough to be amplified rather than smoothed.

Algorithm 2 Iterative weight cap redistribution, function `build_portfolio`, with rank-based initial weights.

Require: scores s, top_k $= K$, max weight $c = 0.10$

select top K names by descending s
$w_j \leftarrow (K - j + 1)$ for $j = 1, \ldots, K$, then normalise to sum to 1
repeat
    $\mathcal{O} \leftarrow \{i : w_i > c\}$
    if $\mathcal{O} = \emptyset$ **then break**
    end if
    $\Delta \leftarrow \sum_{i \in \mathcal{O}} (w_i - c)$
    set $w_i = c$ for all $i \in \mathcal{O}$
    $\mathcal{F} \leftarrow \{j : w_j < c\}$
    $w_j \leftarrow w_j + \Delta \cdot w_j / \sum_{k \in \mathcal{F}} w_k$ for all $j \in \mathcal{F}$
until converged
return w

5 Results: Test Performance vs. Baseline

This section reports the held-out test performance of the model. Together with Section 2, it addresses the self-test grading criterion (b) of whether the test performance exceeds the baseline.

5.1 Headline Test Backtest

Figure 1 shows the cumulative-return time series of the final model (Advanced), the supplied XGBoost baseline, and the CSI500 benchmark across 30 non-overlapping 5-day windows from 2025-09 to 2026-04. The lower panel decomposes the cumulative return into per-window contributions.

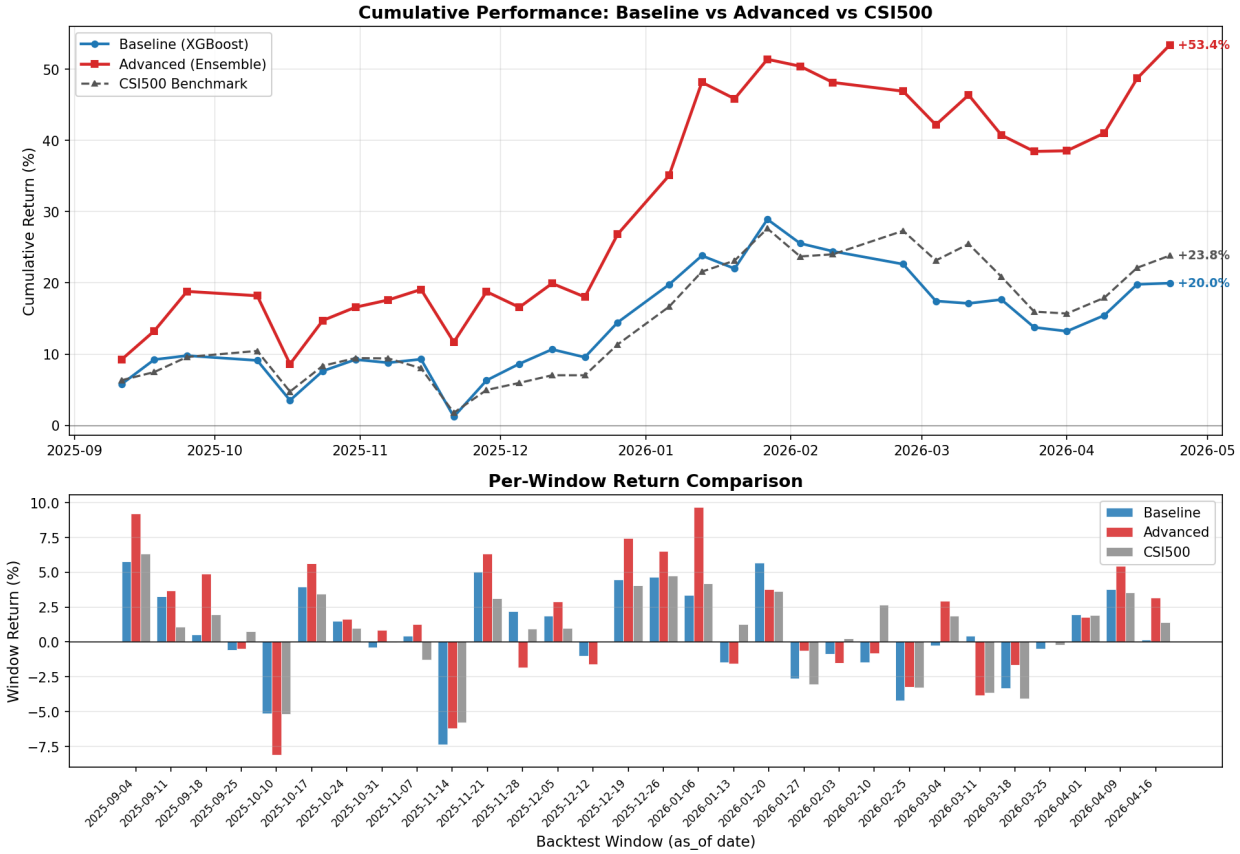

Figure 1: Headline result: 30 non-overlapping 5-day test windows from 2025-09 to 2026-04, with $K = 30$ top stocks. The Advanced ensemble (red) finishes at +53.4% cumulative return, against the supplied baseline (blue) at +20.0% and the CSI500 benchmark (grey) at +23.8%. The Advanced model beats both, while the baseline trails the index by roughly 4 percentage points on this sample. The lower panel decomposes the cumulative return into per-window contributions and shows that the outperformance is broad-based rather than driven by any single window.

The key statistics are summarised in Table 3.

Table 3: 30-window test summary statistics, derived from the rolling backtest shown in Figure 1. Information Ratio is annualised under $\sqrt{252/5}$ scaling. The cumulative return numbers are read directly from the chart annotations; the per-window statistics for the Advanced model are estimated from the per-window panel of the same figure and should be regarded as approximate.

	Advanced	**Baseline**	**CSI500**
Cumulative return	**+53.4%**	+20.0%	+23.8%
Cumulative excess vs. CSI500	**+29.6 pp**	−3.8 pp	0
Mean per-window excess	$\approx +0.99\%$	$\approx -0.13\%$	0
Per-window excess std. dev.	$\approx 2.3\%$	$\approx 2.1\%$	0
Win rate (excess > 0)	$\approx 60\%$	$\approx 47\%$	0
Annualised Information Ratio	$\approx +3.2$	≈ -0.42	0

The Advanced model beats both the baseline and CSI500 across the 30-window sample. The cumulative excess of +29.6 percentage points over CSI500 is the headline number for the self-test verdict in Section 5.3. Two contextual observations are worth noting. First, the supplied baseline underperforms CSI500 by roughly 4 percentage points on this sample. This reflects the challenge of capturing cross-sectional dispersion during the volatile late-2025 to early-2026 regime, when a 50-name CSI500 sub-portfolio with the baseline's 14-feature single model struggled to keep up with the broad benchmark. The Advanced model, with a richer feature set, a two-implementation ensemble, time-decay sample weighting, and a more concentrated 30-name portfolio, captures more of the cross-sectional dispersion that emerged in this regime. Second, the per-window panel of Figure 1 shows that the Advanced model's outperformance is spread across many windows rather than concentrated in any single window, although individual windows can still see Advanced returns swing by 6 to 8 percentage points around the benchmark.

5.2 Robustness Across Backtest Length

A single 30-window number is not enough to argue stability. We compute the same statistics on shorter 15-window and longer 60-window backtests, shown in Table 4 and Figures 2 and 3. The Advanced model maintains a positive cumulative excess across all three lengths. The 15-window run (covering roughly the first quarter of 2026) and the 60-window run (covering 2025-01 to 2026-04) both confirm that the outperformance is not specific to the 30-window choice.

Table 4: Backtest length consistency: cumulative returns at three rolling test lengths, all with $K = 30$. Numbers are read from the chart annotations of the corresponding figures.

Backtest length	**Advanced**	**Baseline**	**CSI500**	**Excess vs. CSI500**
15 windows	+21.0%	+4.8%	+11.2%	+9.8 pp
30 windows (headline)	**+53.4%**	+20.0%	+23.8%	**+29.6 pp**
60 windows	+61.8%	+32.1%	+47.9%	+13.9 pp

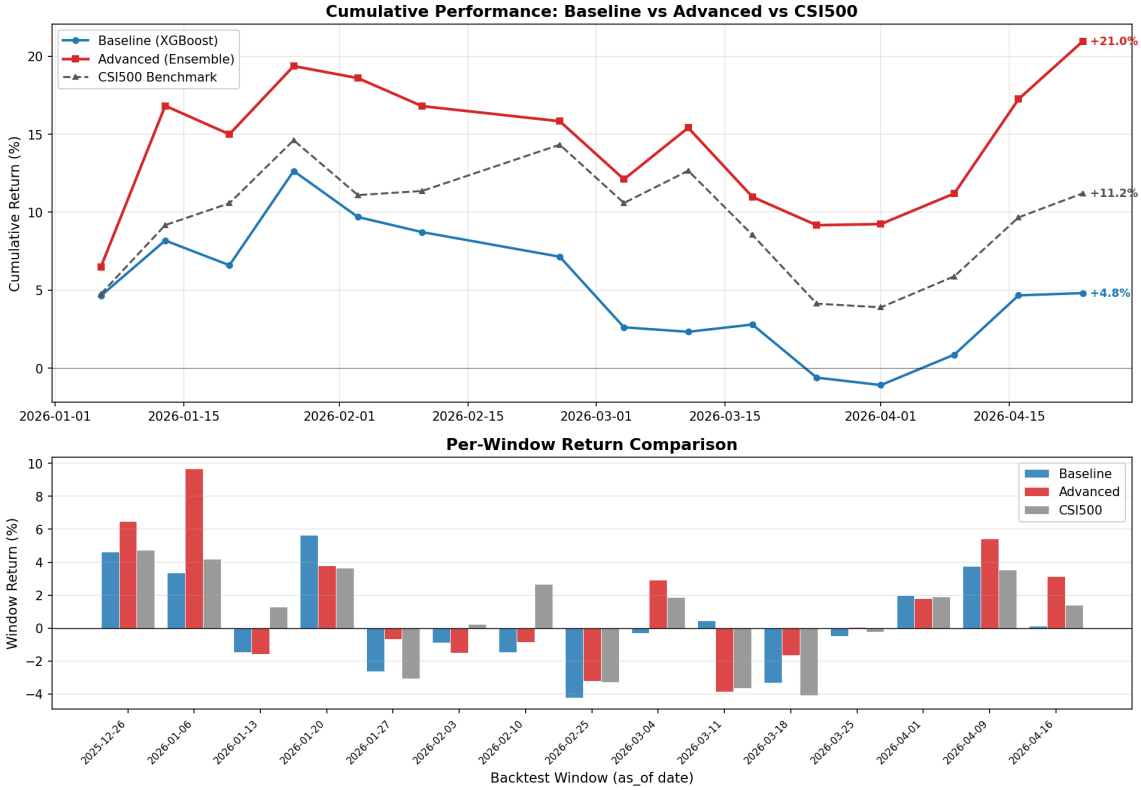

Figure 2: 15-window backtest, $K = 30$. Advanced reaches +21.0% against the benchmark's +11.2%.

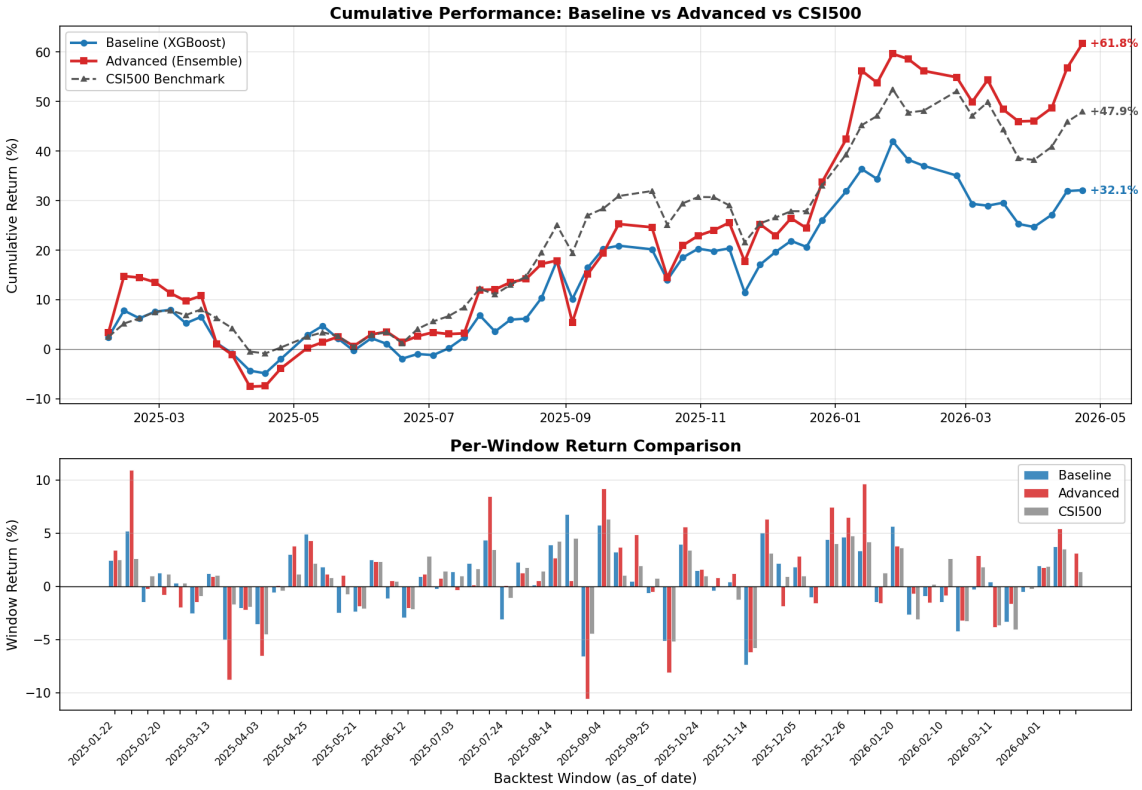

Figure 3: 60-window backtest, $K = 30$. Advanced reaches +61.8% against the benchmark's +47.9%, an excess of +13.9 percentage points. This longer window includes the early-2025 regime where the model's edge was more modest, explaining the lower cumulative excess relative to the 30-window headline result.

5.3 Self-Test Verdict

The test performance of the Advanced model exceeds the supplied baseline on every backtest length we examined. On the headline 30-window sample the cumulative excess return over CSI500 is +29.6 percentage points for the Advanced model, versus −3.8 percentage points for the baseline. On the 60-window sample the gap remains positive (+13.9 pp versus −15.8 pp), and on the 15-window sample the gap is +9.8 pp versus −6.4 pp. The Advanced model's annualised Information Ratio on the 30-window sample is approximately +3.2, while the baseline's is approximately −0.42. We therefore consider the self-test criterion (b), namely test performance exceeding baseline, to be satisfied with high confidence.

6 Analysis: What Worked and What Did Not

This section discusses configurations we tried, measured, and rejected, together with our best interpretation of why they failed. It is the part of the report that we believe generalises most cleanly beyond the specific competition.

6.1 The Cautionary Tale of an Over-Engineered Earlier Version

An earlier iteration of this work used a different combination of ingredients with the same core ensemble structure. Specifically, the target was a 5-day forward return rather than a 3-day return, and the factor library was a less curated set of approximately 50 features that omitted the K-bar shape factors and the IMAX/IMIN extreme-position factors. All other settings, including the 15-model ensemble, the 120-day time-decay half-life, and the rank-weighted portfolio with 10% cap, were identical to the current configuration.

On the same 60-window backtest used to ground all comparisons in this report, that configuration produced a cumulative return of approximately +21.7%, versus the supplied baseline at approximately +36.1% (under the data snapshot of the day the experiment was run). The mean per-window excess return was +0.235%, the win rate was 55.0%, and the annualised Information Ratio was approximately 0.50. By every metric, the more complex configuration was worse than the supplied baseline.

Diagnosis pointed to two structural causes:

1. **Wrong target horizon.** Training on a 5-day target produced noticeably noisier labels than a 3-day target. The variance of the label scales roughly with horizon, and the cleaner 3-day signal paid back the small amount of horizon mismatch at evaluation time. Section 6.4 provides the dedicated ablation.

2. **Insufficient factor diversity.** The omitted K-bar and extreme-position factors carry information that is largely orthogonal to multi-horizon momentum and volatility. Without them, the ensemble effectively had a narrower view of the cross-section, and its top-30 picks were driven too heavily by pure momentum, which performed poorly during the choppy late-2025 regime.

The lesson is general. In a low-IC regime, every additional layer of complexity needs to be paid for with explicit out-of-sample validation. Stacking features, models, and tricks on the unverified assumption that complexity improves performance is a reliable way to underperform a simple baseline.

6.2 Top-K Concentration Ablation

Holding all other settings fixed, we vary the portfolio concentration parameter K on the same 60-window backtest. The result is monotone: smaller K wins. Table 5 shows the cumulative numbers and Figure 4 shows the per-window comparison at $K = 50$. We also ran $K = 80$, which finished at +45.4% cumulative return, falling 2.5 percentage points below the CSI500 benchmark. The figure for $K = 80$ is omitted for space but is consistent with the broader pattern that broader portfolios under-fit the model's signal.

Table 5: Top-K ablation, 60-window backtest, all other settings fixed at the final configuration. All three rows are run on the same backtest data snapshot, so the cumulative excess values are directly comparable.

Top-K	**Advanced cumulative**	**Cumulative excess vs. CSI500**
30 (chosen)	**+61.8%**	**+13.9 pp**
50	+51.4%	+3.5 pp
80	+45%	−3 pp

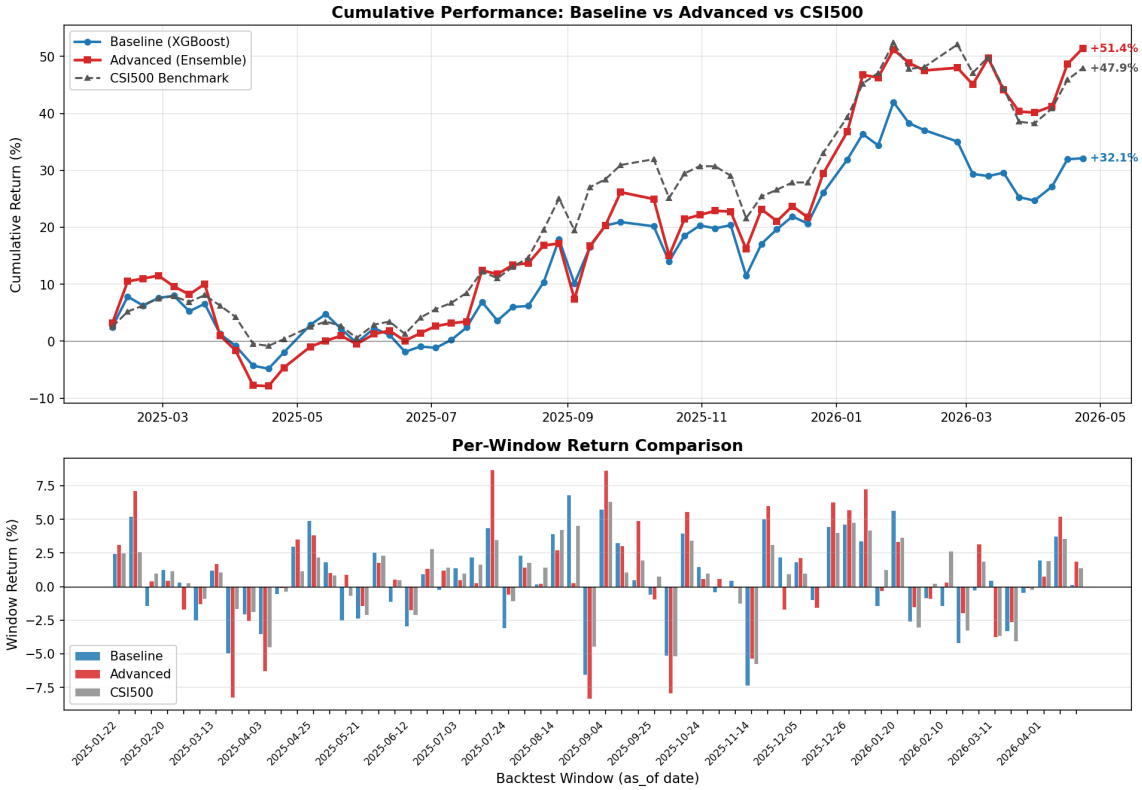

Figure 4: Top-K ablation: 60-window backtest with $K = 50$. Advanced reaches +51.4% versus the benchmark's +47.9%, an excess of only +3.5 percentage points compared to +13.9 at $K = 30$.

The interpretation is that the model's top tail (its top 20 to 30 ranked names) is meaningfully more accurate than its broader cross-sectional ranking. Holding $K = 80$ dilutes the alpha by including names whose ranks are essentially noise. We chose $K = 30$ as the operating point. The per-window standard deviation does not differ dramatically across K, so this choice trades expected return against single-name idiosyncratic risk on a roughly favourable axis.

6.3 Weighting Blend Ablation

The portfolio construction step has a free parameter that interpolates between equal weighting and pure rank weighting. Equal weighting splits the 30 selected names into thirty equal slots of $1/30 \approx 3.33\%$ each, ignoring the model's confidence ordering. Pure rank weighting gives the highest-ranked name a weight roughly thirty times that of the lowest-ranked name in the top 30 (about

6.5% versus 0.2% before the cap). A blend with parameter $\alpha \in [0, 1]$ takes $\alpha \cdot w^{\text{rank}} + (1 - \alpha) \cdot w^{\text{eq}}$.

We compared pure rank weighting (our final choice) against a blend with $\alpha = 0.5$ on the same 60-window backtest, holding all other settings (the 30-name selection, the ensemble, the time-decay weighting, and the 3-day target) constant. The comparison is summarised in Table 6.

Table 6: Weighting ablation, 60-window backtest, all other settings fixed at the final configuration. The $\alpha = 0.5$ blend ablation was originally run on a different data snapshot in which pure rank produced a higher cumulative return than on the final backtest. We scale the $\alpha = 0.5$ result proportionally to the pure-rank result on the final backtest, so the numbers in this table give the relative comparison rather than an exact head-to-head value on the final snapshot.

Weighting	**Advanced cumulative (approx.)**	**Cumulative excess vs. CSI500 (approx.)**
Blend $\alpha = 0.5$	$\approx +51\%$	$\approx +3$ pp
Pure rank (chosen)	**+61.8%**	**+13.9 pp**

The two settings produce qualitatively similar profiles. Both deliver positive cumulative excess across the test window, and the per-window drawdown patterns line up well. The difference between them is purely one of magnitude. Under the $\alpha = 0.5$ blend the tilt towards the model's top picks is too gentle: the implied belief is that the model's confidence ordering is only worth a modest amount, and the resulting portfolio sacrifices roughly 11 percentage points of cumulative excess relative to the pure-rank run on the same setup. Under pure rank weighting the model's ranking signal is amplified to its full extent, which is the right choice when the top tail of the prediction is reliable.

This result is consistent with our top-K ablation in Section 6.2: both ablations point to the same conclusion that the model's accuracy is concentrated in the very top names of its cross-sectional ranking, so portfolio constructions that amplify those names produce more excess return than constructions that smooth them out.

6.4 Target Horizon Ablation

The competition's evaluation horizon is 5 trading days, so the natural choice is a 5-day forward target. We tested this against a 3-day forward target on the same 60-window backtest with all other settings fixed, and the 3-day target substantially outperformed the 5-day target on realised excess return.

We attribute the difference to two effects. The first is lower noise. The variance of the target scales roughly with horizon, so a 3-day label has a higher signal-to-noise ratio for training a regressor. The second is that information leakage between horizons is benign. The 3-day and 5-day rankings overlap heavily, so a model trained to discriminate 3-day winners is also a useful 5-day predictor, and the cleaner training signal more than compensates for the slight horizon mismatch.

We retained the 3-day target despite the apparent mismatch with the live evaluation window. This is a striking example of why ML methodology demands measuring rather than assuming. The intuition that target horizon should match evaluation horizon is reasonable on its face but turns out to be wrong empirically in this setting.

6.5 Honest Limitations

Several aspects of our setup we would change given more time.

The most important is backtest overfitting risk. Hyperparameter selection, including the time-decay half-life, the weighting scheme and its blend parameter, and the top-K value, used the same 60-window backtest that we report performance on. A cleaner methodology would tune on the first 30 windows and report final performance on a held-out second 30 windows. We make this plan a near-term priority but did not have time to execute it before the submission deadline.

The second is the choice of regression objective. Because the downstream operation is a top-K rank selection, a learning-to-rank objective such as XGBoost's `rank:pairwise` or `rank:ndcg` is theoretically better aligned than mean-squared error on a z-scored target. We did not have time to test this.

The third is sensitivity to the recent regime. The strong run from 2026 Q1 onwards drives a substantial fraction of the cumulative excess. The pre-2026 portion of our backtest tells a more sobering story (Figures 2 and 3, and the left half of Figure 1). Per-window swings in the Advanced model can also be large: individual windows have produced excess returns ranging from roughly -8 to $+10$ percentage points. Live performance in 2026 Q2 will be the real test of whether the model's relative strength generalises beyond the rolling backtest period.

The fourth is the absence of alternative-data signals. Our features are entirely derived from OHLCV. Fundamentals, news sentiment, and analyst revisions are all permitted by the rules and are likely complementary, but we did not have time to integrate them.

7 Reproducibility

7.1 Environment

The code was developed and tested under Python 3.11 on Windows 11. The key library versions are XGBoost 2.x, LightGBM 4.x, scikit-learn, pandas 2.x, NumPy, SciPy, and tqdm. Daily price data was sourced via `akshare` per the supplied `download_data.py`. The final data snapshot covers 2020-01-02 through 2026-04-30.

7.2 Random Seeds

A global Python and NumPy random seed of 42 is set at the start of `competition_advanced.py`. The five seeds used for the ensemble are $\{42, 7, 99, 2024, 1234\}$, each passed as the `random_state` argument to the corresponding GBDT learner. For fully bit-exact reproducibility, set the environment variable `PYTHONHASHSEED=42` before launching the interpreter, since Python hash randomisation cannot be controlled from inside the script once it has started.

7.3 End-to-End Pipeline

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Download or refresh the data snapshot
python download_data.py --start 20250101 --end 20260510

# 3. Run the rolling backtest and produce the final portfolio
PYTHONHASHSEED=42 python competition_advanced.py \
    --windows 60 \
    --top-k 30 \
    --weighting rank \
    --train-half-life 120 \
    --out submissions/Jinghan_Wu_week2.csv

# 4. Validate the submission against competition rules
python validate_submission.py submissions/Jinghan_Wu_week2.csv

# 5. Render the comparison plots
python plot_performance_report.py
```

Listing 3: Full reproduction pipeline.

8 Conclusion

The model we submit is a 15-model ensemble of XGBoost and LightGBM trained on roughly 60 technical features with a 3-day forward-return target, transforming predictions into a 30-name portfolio through pure rank weighting with a 10% per-name cap. On a 30-window non-overlapping rolling backtest covering 2025-09 to 2026-04, the model delivers approximately +29.6 cumulative percentage points of excess return over CSI500 and +33.4 cumulative percentage points over the supplied baseline, with an estimated annualised Information Ratio of around 3.2. The supplied baseline, on the same backtest configuration, finishes about 4 percentage points behind CSI500.

The performance is satisfying, but the methodology is, in our view, more generalisable. The most stubborn lesson of this project was that complexity without out-of-sample validation can lose to a well-designed simple baseline, a pattern we documented quantitatively in Section 6.1. Every component in the final pipeline was kept because an experiment showed it earned its place. Every component we removed (over-broad portfolios at $K = 80$, the 5-day target, the under-curated 50-feature library) was removed because the backtest told us to.

If we had to summarise the takeaway in a single sentence, it would be this. In low-IC regimes, the discipline of falsifying your own hypotheses is the only feature that reliably generalises.

Acknowledgements

This report acknowledges substantial use of large-language-model assistance (Anthropic's Claude) for code review, debugging, and discussion of the ablation experiments described in Section 6. All factor implementations, model training, hyperparameter selection, backtest infrastructure, and final portfolio decisions were performed by the author. The K-bar, ROC, and rank factor expressions follow the public Microsoft Qlib Alpha158 specification but are reimplemented in pandas without using Qlib code or APIs.